

Project: Build a Data Mart in SQL

Course code: DLBDSPBDM01

Use case: Airbnb Accommodation Booking Platform

Field	Detail
Course of Study	BSc. Computer Science.
Student's Name	Kashish Paresh Gada
Matriculation No.	102202657
Tutor	Anna Androvitsanea
Submission	Conception Phase (Phase 1)
Date	7th July 2026

1. Problem Definition and Objective

Airbnb is an online accommodation platform that connects host who wants to offer lodging with guests who want to book it for staying temporarily while the platform itself handles all the booking and financial settlements between both of them that is the host and the user or the guest. Operating such an Airbnb service generates a lot of interrelated data like user profiles, accommodation, listings, availability, bookings, payments, payouts, reviews and messages amongst others. To manage this information in a non-relational way quickly leads to data redundancy update anomalies and inconsistent records which will undermine billing accuracy, along with reduced trust between users and the reliability of the service.

The main goal of this portfolio is to design and implement A normalized relational database that models the Airbnb use case end to end this conception phase establishes the foundation for further building upon. It captures the requirements of the systems and identifies the roles involved and actions they perform and translates these into a formal entity relationship model. We also have a data dictionary that explains what each entity does.

2. Requirements Analysis

2.1 Roles (User Groups)

Every person on the platform holds a single User account. An account can act in one or both of two functional roles. Platform-level tasks (collecting payment, releasing the payout to the host) are performed automatically by the system rather than by an end user.

Role	Description and responsibilities
Guest	A user who searches for and books accommodation, pays for the stay, communicates with hosts, leaves reviews and saves listings to wishlists.
Host	A user who lists and manages accommodation, sets prices, availability, amenities, photos, house rules and a cancellation policy, receives bookings and payouts, and reviews guests.
User (shared)	The common account from which Host and Guest are specialised; holds identity, contact and login data and may link social-network profiles. A single user can be both a host and a guest.

2.2 Actions Performed by Each Role

Guest actions:

- Guests can register an account and manage their personal profile and add social links.
- Guests can search and filter listings by their location, price, room type and amenity requirements.
- They can view the listings availability calendar and house rules and check if they fit them or not.
- Guests can create a booking for a date range and number of guests that are going to stay on the Airbnb.
- Guests can pay for the booking using a stored payment method.
- Messages can be exchanged with the host about a booking.
- The guest can write a review and add a rating after the stay.
- If a guest likes a property and wants to save that for future they can save it in the wish lists.
-

Host actions:

- A user can register as a host and maintain the host profile.
- Hosts can create and edit listings they can also include address property and room types.
- Hosts can manage availability calendars, prices, amenities photos and house rules so the guest can view it.
- Hosts can also add a cancellation policy to each listing.
- Hosts can receive and track the bookings and check the payouts.
- Hosts can communicate and message the guests and review them after a stay.
- Hosts can estimate and expected income from a listing.

2.3 Required Data and Functions

Required data:

- Account and identity data for users, hosts and guests, plus linked social profiles and stored payment/payout methods.
- Listing data with location (country, city, address), property and room type, amenities, photos, house rules and a per-date availability calendar.
- Transactional data: bookings, payments, host payouts and the fees/commissions charged on each booking.
- Feedback data: reviews, the users they concern and per-category ratings.
- Communication data: direct messages between users.

Required functions:

- Account and profile management for both roles.
- Listing search, filtering and availability checking.
- The full booking lifecycle (request, confirm, cancel, complete).
- Payment capture from the guest and payout to the host after arrival, with commission/fee handling.
- Mutual guest-host reviews and rating aggregation.
- Messaging and display of a user's reputation through social contacts.

3. Entity Relationship Model

My ER diagram below (drawn using draw.io) presents the complete ER model as a relational schema. Each entity is shown as a table listing its attributes with the MySQL data type; primary keys are marked PK (underlined) and foreign keys FK. Relationships are drawn as connectors using crow's-foot cardinality, where a single bar denotes the “one” side and a fork denotes the “many” side. The model contains 25 entities, which together with the four junction tables that resolve the many-to-many and recursive relationships yields 29 tables at implementation.

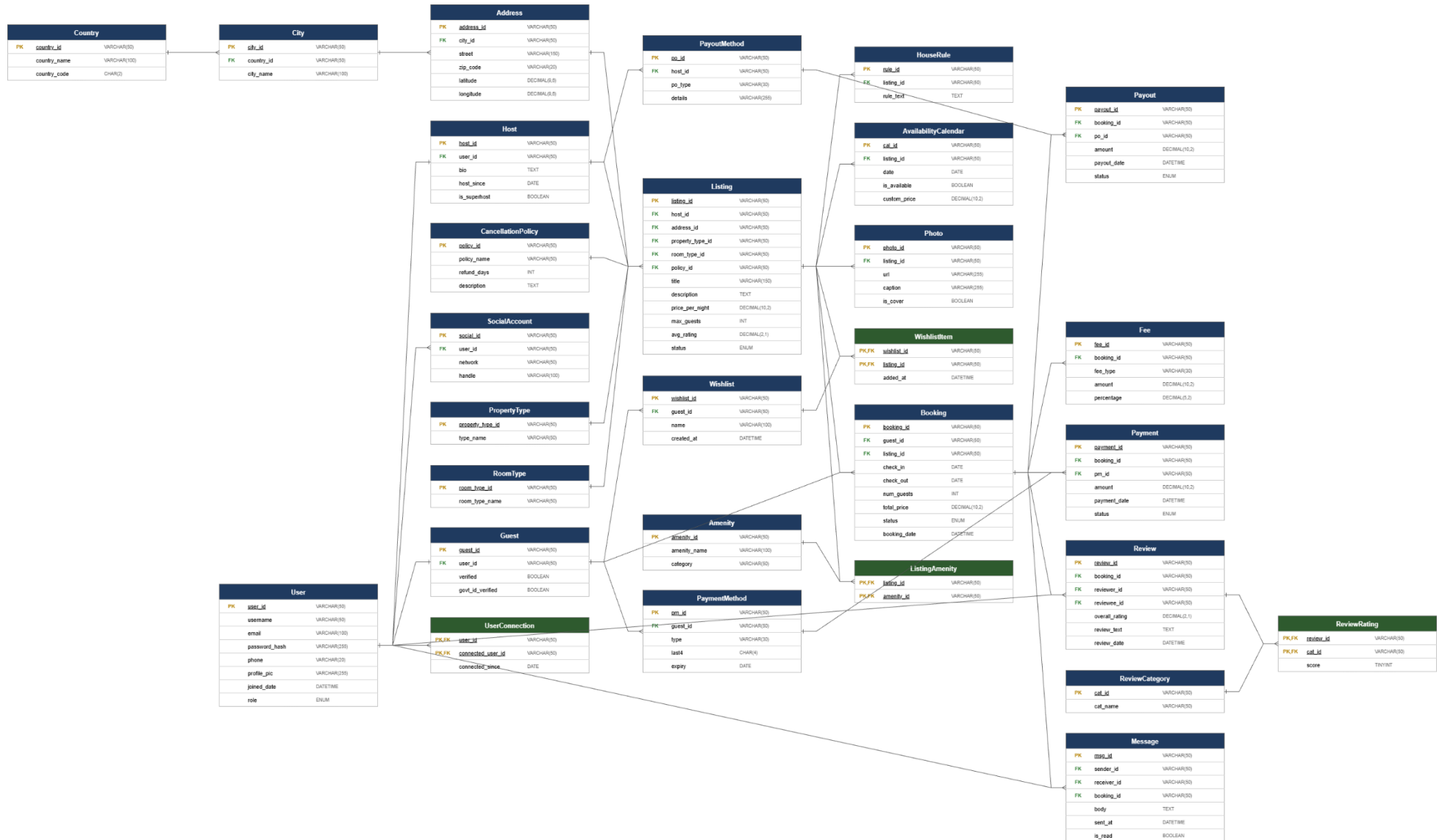


Figure 1: Entity Relationship Model (ERM)¹ – relational schema (crow's-foot notation)

¹ Self-drawn on draw.io

Triple (ternary) relationships – joins over three tables:

- books: a Guest books a Listing for dates held in the AvailabilityCalendar, realised through the Booking entity (Guest – Booking – Listing).
- reviews: a Review links a Booking to the reviewer and the reviewee, both of which are Users (User – Review – Booking).
- A third three-table join arises through Payment, which connects a Booking to the PaymentMethod used.

Recursive relationship:

- connected to: a User may be connected to other Users (social contacts), a many-to-many relationship of the User entity with itself, resolved by the UserConnection junction table.

Normalisation:

- The schema is normalised to 3NF. Look-up data (country, city, property type, room type, cancellation policy, amenity, review category) is factored into its own tables, and every many-to-many relationship is resolved with a junction table so that no repeating groups or transitive dependencies remain.

4. Data Dictionary

The data dictionary lists each entity with its attributes, MySQL data types and a short description. Primary keys are marked PK and foreign keys FK.

User

Base account shared by every person on the platform. Specialised into Host and Guest.

Attribute	Data Type	Key	Description
user_id	VARCHAR(50)	PK	Unique identifier for each user account.
username	VARCHAR(50)		Unique public display / login name.
email	VARCHAR(100)		Unique contact e-mail address.
password_hash	VARCHAR(255)		Securely hashed account password.
phone	VARCHAR(20)		Contact phone number.
profile_pic	VARCHAR(255)		Path/URL of profile picture.

Attribute	Data Type	Key	Description
joined_date	DATETIME		Date the account was created.
role	ENUM('GUEST','HOST','BOTH')		Indicates how the account is used.

Host

Role specialisation of User who lists accommodation.

Attribute	Data Type	Key	Description
host_id	VARCHAR(50)	PK	Unique host identifier.
user_id	VARCHAR(50)	FK	References the underlying User account.
bio	TEXT		Self-description shown on the host profile.
host_since	DATE		Date the user became a host.
is_superhost	BOOLEAN		Whether the host has Superhost status.

Guest

Role specialisation of User who books accommodation.

Attribute	Data Type	Key	Description
guest_id	VARCHAR(50)	PK	Unique guest identifier.
user_id	VARCHAR(50)	FK	References the underlying User account.
verified	BOOLEAN		Whether the guest identity is verified.
govt_id_verified	BOOLEAN		Whether a government ID was confirmed.

SocialAccount

External social-network profile linked to a user (Facebook etc.).

Attribute	Data Type	Key	Description
social_id	VARCHAR(50)	PK	Unique identifier of the linked profile.
user_id	VARCHAR(50)	FK	User who owns the linked profile.
network	VARCHAR(50)		Name of the social network.

Attribute	Data Type	Key	Description
handle	VARCHAR(100)		Profile handle or link.

PaymentMethod

A guest's stored means of payment.

Attribute	Data Type	Key	Description
pm_id	VARCHAR(50)	PK	Unique payment-method identifier.
guest_id	VARCHAR(50)	FK	Guest who owns the payment method.
type	VARCHAR(30)		e.g. credit card, PayPal.
last4	CHAR(4)		Last four digits (display only).
expiry	DATE		Expiry date of the instrument.

PayoutMethod

A host's account used to receive payouts.

Attribute	Data Type	Key	Description
po_id	VARCHAR(50)	PK	Unique payout-method identifier.
host_id	VARCHAR(50)	FK	Host who owns the payout method.
po_type	VARCHAR(30)		e.g. bank transfer, PayPal.
details	VARCHAR(255)		Masked account details / IBAN.

Country

Reference table of countries for normalised location data.

Attribute	Data Type	Key	Description
country_id	VARCHAR(50)	PK	Unique country identifier.
country_name	VARCHAR(100)		Full country name.
country_code	CHAR(2)		ISO 3166-1 alpha-2 code.

City

Reference table of cities, each belonging to one country.

Attribute	Data Type	Key	Description
city_id	VARCHAR(50)	PK	Unique city identifier.
country_id	VARCHAR(50)	FK	Country the city belongs to.
city_name	VARCHAR(100)		Name of the city.

Address

Physical address of a listing, linked to a city.

Attribute	Data Type	Key	Description
address_id	VARCHAR(50)	PK	Unique address identifier.
city_id	VARCHAR(50)	FK	City of the address.
street	VARCHAR(150)		Street and house number.
zip_code	VARCHAR(20)		Postal code.
latitude	DECIMAL(9,6)		Geographic latitude.
longitude	DECIMAL(9,6)		Geographic longitude.

Listing

An accommodation offered by a host (the central entity).

Attribute	Data Type	Key	Description
listing_id	VARCHAR(50)	PK	Unique listing identifier.
host_id	VARCHAR(50)	FK	Host who owns the listing.
address_id	VARCHAR(50)	FK	Address of the listing.
property_type_id	VARCHAR(50)	FK	Type of property.
room_type_id	VARCHAR(50)	FK	Type of room offered.
policy_id	VARCHAR(50)	FK	Cancellation policy that applies.
title	VARCHAR(150)		Headline shown to guests.
description	TEXT		Full textual description.

Attribute	Data Type	Key	Description
price_per_night	DECIMAL(10,2)		Base nightly price.
max_guests	INT		Maximum number of guests.
avg_rating	DECIMAL(2,1)		Average review rating.
status	ENUM('ACTIVE','INACTIVE')		Whether the listing is bookable.

PropertyType

Look-up of property types (apartment, house, loft, ...).

Attribute	Data Type	Key	Description
property_type_id	VARCHAR(50)	PK	Unique property-type identifier.
type_name	VARCHAR(50)		Name of the property type.

RoomType

Look-up of room types (entire place, private, shared).

Attribute	Data Type	Key	Description
room_type_id	VARCHAR(50)	PK	Unique room-type identifier.
room_type_name	VARCHAR(50)		Name of the room type.

CancellationPolicy

Cancellation/refund rules referenced by listings.

Attribute	Data Type	Key	Description
policy_id	VARCHAR(50)	PK	Unique policy identifier.
policy_name	VARCHAR(50)		Name (Flexible, Moderate, Strict).
refund_days	INT		Days before check-in for full refund.
description	TEXT		Human-readable policy text.

Amenity

Catalogue of amenities (Wi-Fi, pool, kitchen, ...).

Attribute	Data Type	Key	Description
amenity_id	VARCHAR(50)	PK	Unique amenity identifier.
amenity_name	VARCHAR(100)		Name of the amenity.
category	VARCHAR(50)		Grouping (e.g. Safety, Kitchen).

Photo

Photographs attached to a listing.

Attribute	Data Type	Key	Description
photo_id	VARCHAR(50)	PK	Unique photo identifier.
listing_id	VARCHAR(50)	FK	Listing the photo belongs to.
url	VARCHAR(255)		Path/URL of the image.
caption	VARCHAR(255)		Optional caption.
is_cover	BOOLEAN		Whether it is the cover photo.

HouseRule

House rules defined per listing.

Attribute	Data Type	Key	Description
rule_id	VARCHAR(50)	PK	Unique rule identifier.
listing_id	VARCHAR(50)	FK	Listing the rule applies to.
rule_text	TEXT		Description of the rule.

AvailabilityCalendar

Per-date availability and price overrides for a listing.

Attribute	Data Type	Key	Description
cal_id	VARCHAR(50)	PK	Unique calendar-entry identifier.
listing_id	VARCHAR(50)	FK	Listing the entry refers to.

Attribute	Data Type	Key	Description
date	DATE		Calendar date.
is_available	BOOLEAN		Whether the date is bookable.
custom_price	DECIMAL(10,2)		Optional price override.

Booking

A reservation of a listing by a guest (associative entity).

Attribute	Data Type	Key	Description
booking_id	VARCHAR(50)	PK	Unique booking identifier.
guest_id	VARCHAR(50)	FK	Guest who made the booking.
listing_id	VARCHAR(50)	FK	Listing that was booked.
check_in	DATE		Arrival date.
check_out	DATE		Departure date.
num_guests	INT		Number of guests.
total_price	DECIMAL(10,2)		Total amount for the stay.
status	ENUM('PENDING','CONFIRMED','CANCELLED','COMPLETED')		Booking lifecycle state.
booking_date	DATETIME		When the booking was made.

Payment

Payment made by a guest for a booking.

Attribute	Data Type	Key	Description
payment_id	VARCHAR(50)	PK	Unique payment identifier.
booking_id	VARCHAR(50)	FK	Booking being paid for.
pm_id	VARCHAR(50)	FK	Payment method used.
amount	DECIMAL(10,2)		Amount paid.
payment_date	DATETIME		Date of payment.
status	ENUM('PAID','REFUNDED','FAILED')		Payment state.

Payout

Disbursement to the host (released after arrival).

Attribute	Data Type	Key	Description
payout_id	VARCHAR(50)	PK	Unique payout identifier.
booking_id	VARCHAR(50)	FK	Booking being settled.
po_id	VARCHAR(50)	FK	Payout method used.
amount	DECIMAL(10,2)		Amount paid to host.
payout_date	DATETIME		Date the payout was released.
status	ENUM('PENDING','RELEASED')		Payout state.

Fee

Commission/fee lines attached to a booking.

Attribute	Data Type	Key	Description
fee_id	VARCHAR(50)	PK	Unique fee identifier.
booking_id	VARCHAR(50)	FK	Booking the fee relates to.
fee_type	VARCHAR(30)		Guest service / host service / cleaning.
amount	DECIMAL(10,2)		Fee amount.
percentage	DECIMAL(5,2)		Rate applied, where relevant.

Review

A review written about a stay (associative; links two users).

Attribute	Data Type	Key	Description
review_id	VARCHAR(50)	PK	Unique review identifier.
booking_id	VARCHAR(50)	FK	Booking being reviewed.
reviewer_id	VARCHAR(50)	FK	User who wrote the review.
reviewee_id	VARCHAR(50)	FK	User being reviewed.
overall_rating	DECIMAL(2,1)		Overall score (1.0-5.0).
review_text	TEXT		Free-text comment.
review_date	DATETIME		Date the review was posted.

ReviewCategory

Categories scored within a review (cleanliness, ...).

Attribute	Data Type	Key	Description
cat_id	VARCHAR(50)	PK	Unique category identifier.
cat_name	VARCHAR(50)		Name of the category.

Message

Direct message exchanged between two users.

Attribute	Data Type	Key	Description
msg_id	VARCHAR(50)	PK	Unique message identifier.
sender_id	VARCHAR(50)	FK	User who sent the message.
receiver_id	VARCHAR(50)	FK	User who received it.
booking_id	VARCHAR(50)	FK	Related booking (nullable).
body	TEXT		Message content.
sent_at	DATETIME		Timestamp of sending.
is_read	BOOLEAN		Whether it has been read.

Wishlist

A guest's saved collection of listings.

Attribute	Data Type	Key	Description
wishlist_id	VARCHAR(50)	PK	Unique wishlist identifier.
guest_id	VARCHAR(50)	FK	Guest who owns the wishlist.
name	VARCHAR(100)		Wishlist name.
created_at	DATETIME		Date created.

Associative / junction tables (created during implementation)

The following tables resolve the many-to-many and recursive relationships shown in the ERM.

ListingAmenity

Resolves the M:N relationship between Listing and Amenity.

Attribute	Data Type	Key	Description
listing_id	VARCHAR(50)	PK, FK	Listing offering the amenity.
amenity_id	VARCHAR(50)	PK, FK	Amenity offered.

ReviewRating

Resolves the M:N relationship between Review and ReviewCategory.

Attribute	Data Type	Key	Description
review_id	VARCHAR(50)	PK, FK	Review being scored.
cat_id	VARCHAR(50)	PK, FK	Category scored.
score	TINYINT		Score for that category (1-5).

WishlistItem

Resolves the M:N relationship between Wishlist and Listing.

Attribute	Data Type	Key	Description
wishlist_id	VARCHAR(50)	PK, FK	Wishlist containing the listing.
listing_id	VARCHAR(50)	PK, FK	Listing saved.
added_at	DATETIME		When it was added.

UserConnection

Resolves the recursive M:N 'connected to' relationship on User (social contacts).

Attribute	Data Type	Key	Description
user_id	VARCHAR(50)	PK, FK	First user.
connected_user_id	VARCHAR(50)	PK, FK	Connected user.
connected_since	DATE		Date the connection was made.

PHASE 2: DEVELOPMENT PHASE

Build a Data Mart in SQL

Airbnb Accommodation Booking Platform

COURSE

DLBDSPBDM01

NAME

Kashish Paresh Gada

MATRICULATION NO.

102202657

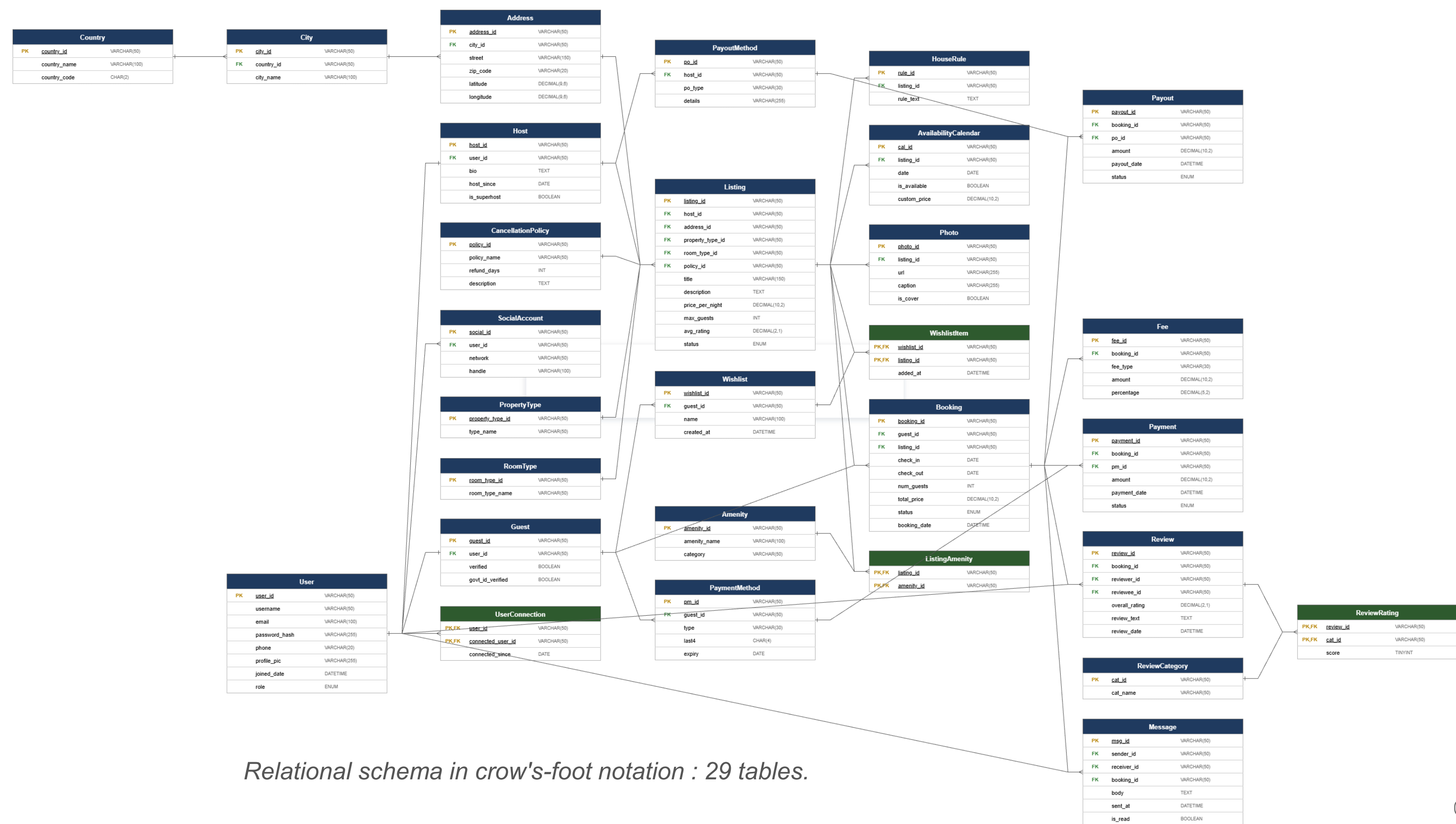
Submission Date

20th July 2026

Project Overview

- Goal: a fully normalised relational database for an Airbnb style accommodation booking platform.
- Models users, listings, bookings, payments, payouts, reviews and messaging.
- 25 entities + 4 junction tables = 29 tables in total.
- Implemented and tested end to end in MySQL 8.

Database Design - ER Model



Relational schema in crow's-foot notation : 29 tables.

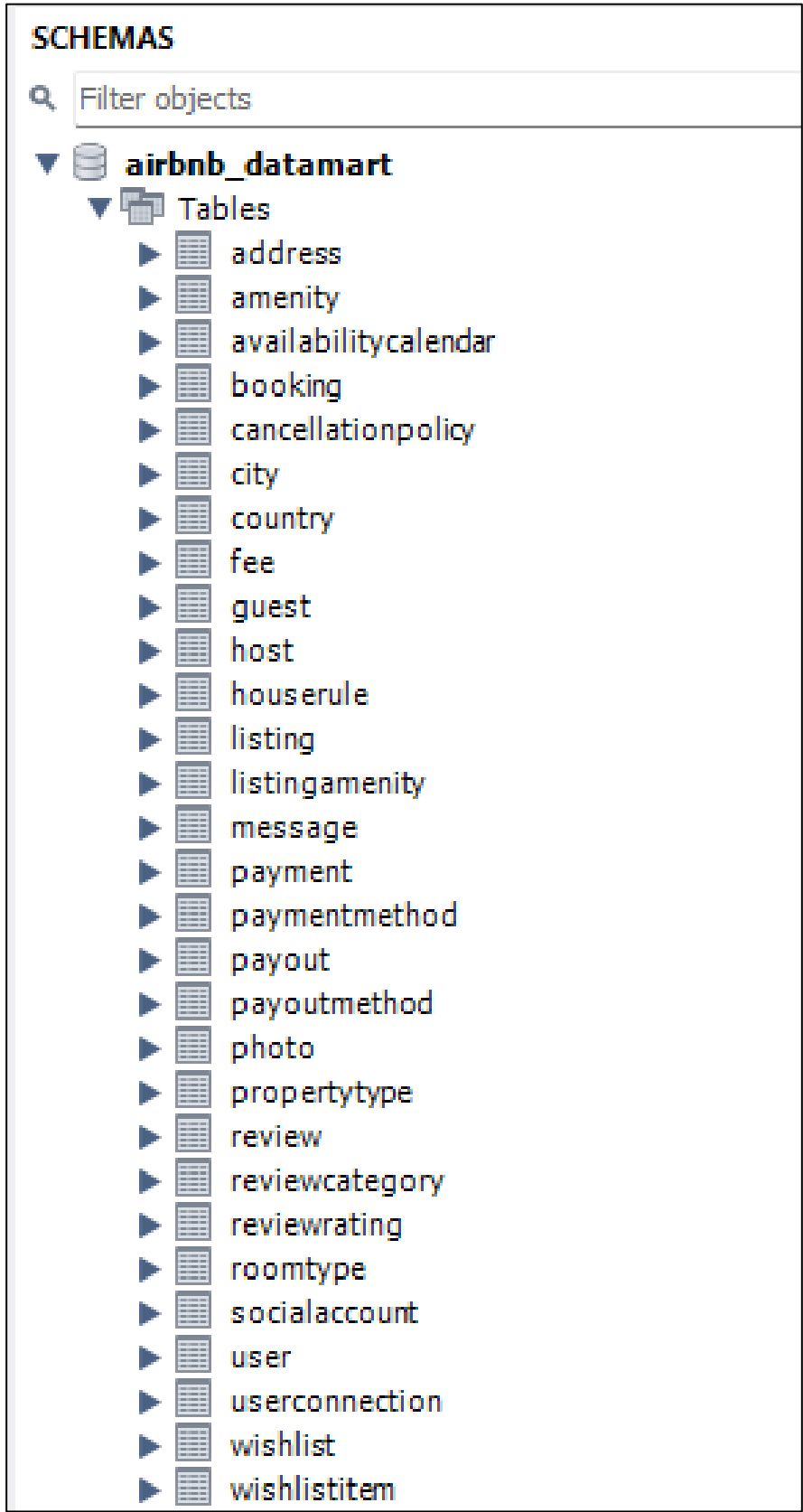
Implementation Approach

- MySQL 8 with the InnoDB storage engine for full referential integrity.
- Readable VARCHAR IDs for clear, human-legible keys.
- Every relationship enforced with a FOREIGN KEY.
- Business rules enforced with CHECK constraints.
- UNIQUE constraints for identity fields and 1:1 relationships.
- Tables created parent-first, so the script runs cleanly top to bottom.

Constraints Used

Constraint	Count	Purpose
PRIMARY KEY	29	One per table.
FOREIGN KEY	37	Every relationship.
UNIQUE	16	Emails, usernames, 1:1 links.
CHECK	19	Ratings 1-5, valid dates, non-negative money, no self-references.

Database Built - Schema



All 29 tables created successfully in MySQL Workbench.

Example Table Definition

```
-- =====
-- 20. BOOKING -- reservation of a listing by a guest
-- =====

• CREATE TABLE `booking` (
  `booking_id`  VARCHAR(50)  NOT NULL,           -- PK
  `guest_id`    VARCHAR(50)  NOT NULL,           -- FK -> guest
  `listing_id`  VARCHAR(50)  NOT NULL,           -- FK -> listing
  `check_in`    DATE         NOT NULL,
  `check_out`   DATE         NOT NULL,
  `num_guests`  INT          NOT NULL DEFAULT 1,
  `total_price` DECIMAL(10,2) NOT NULL,
  `status`      ENUM('PENDING','CONFIRMED','CANCELLED','COMPLETED')
                NOT NULL DEFAULT 'PENDING',
  `booking_date` DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,

  CONSTRAINT `pk_booking` PRIMARY KEY (`booking_id`),
  CONSTRAINT `fk_booking_guest`
    FOREIGN KEY (`guest_id`) REFERENCES `guest` (`guest_id`)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT `fk_booking_listing`
    FOREIGN KEY (`listing_id`) REFERENCES `listing` (`listing_id`)
    ON UPDATE CASCADE ON DELETE RESTRICT,
  CONSTRAINT `chk_booking_dates` CHECK (`check_out` > `check_in`),
  CONSTRAINT `chk_booking_guests` CHECK (`num_guests` > 0),
  CONSTRAINT `chk_booking_total` CHECK (`total_price` >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

The Booking table showing PK, FKs and CHECK constraints (check_out > check_in).

Test Cases Overview

- 13 SQL test cases validate the design end to end.
- Cover the full business flow: booking → payment → payout → review → messaging.
- Include the ternary and recursive relationships in the model.

End-to-End Booking Scenario

Traces a booking across guest, listing, host, payment and payout (joins 8 tables).

10

-- -----

11

-- TEST CASE 1 (primary): end-to-end booking scenario.

12

-- Traces a booking across guest -> user, listing -> host -> user,

13

-- payment and payout. Proves the core business flow is consistent.

14

-- Joins 8 tables.

15

-- -----

16

• SELECT b.booking_id,

17

gu.username AS guest,

18

l.title AS listing,

19

hu.username AS host,

20

b.check_in, b.check_out, b.total_price,

21

b.status AS booking_status,

22

pay.amount AS amount_paid,

23

pay.status AS payment_status,

24

po.amount AS host_payout,

25

po.status AS payout_status

26

FROM booking b

27

JOIN guest g ON b.guest_id = g.guest_id

28

JOIN `user` gu ON g.user_id = gu.user_id

29

JOIN listing l ON b.listing_id = l.listing_id

30

JOIN host h ON l.host_id = h.host_id

31

JOIN `user` hu ON h.user_id = hu.user_id

32

LEFT JOIN payment pay ON pay.booking_id = b.booking_id

33

LEFT JOIN payout po ON po.booking_id = b.booking_id

34

ORDER BY b.booking_id

35

LIMIT 10;

36

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	booking_id	guest	listing	host	check_in	check_out	total_price	booking_status	amount_paid	payment_status	host_payout	payout_status
▶	BKG001	ines.bakker21	Cosy city-centre apartment	liam.jansen1	2026-04-03	2026-04-08	1130.90	COMPLETED	1266.61	PAID	1096.97	RELEASED
	BKG002	marta.weber22	Bright loft with skyline view	emma.muller2	2026-11-20	2026-11-24	876.24	COMPLETED	981.39	PAID	849.95	RELEASED
	BKG003	pavel.bernard23	Charming canal house	noah.dubois3	2026-09-14	2026-09-18	1227.20	COMPLETED	1374.46	PAID	1190.38	RELEASED
	BKG004	sofia.lopez24	Modern studio near park	olivia.garcia4	2026-01-14	2026-01-18	427.12	CONFIRMED	478.37	PAID	414.31	PENDING
	BKG005	hugo.bianchi25	Spacious family villa	ava.rossi5	2026-08-05	2026-08-12	403.97	COMPLETED	452.45	PAID	391.85	RELEASED
	BKG006	elena.costa26	Quiet garden retreat	elijah.silva6	2026-05-20	2026-05-27	1794.31	PENDING	2009.63	FAILED	1740.48	PENDING
	BKG007	anton.brown27	Stylish penthouse suite	sophia.smith7	2026-10-09	2026-10-16	1112.79	CONFIRMED	1246.32	PAID	1079.41	PENDING
	BKG008	klara.kelly28	Rustic countryside cottage	lucas.byrne8	2026-04-15	2026-04-20	1212.75	COMPLETED	1358.28	PAID	1176.37	RELEASED
	BKG009	bjorn.wouters29	Sunny balcony flat	mia.peeters9	2026-03-16	2026-03-20	448.40	COMPLETED	502.21	PAID	434.95	RELEASED
	BKG010	aolife.berger30	Designer duplex downtown	mason.gruber10	2026-05-20	2026-05-24	1161.92	COMPLETED	1301.35	PAID	1127.06	RELEASED

Fee Breakdown

Reconciles the guest service fee (12%) and host service fee (3%) per booking.

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

-- -----

-- TEST CASE 2: fee breakdown per booking (guest 12% + host 3%),

-- reconciling what the guest paid vs. what the host receives.

-- -----

SELECT b.booking_id, b.total_price,

MAX(CASE WHEN f.fee_type='Guest service fee' THEN f.amount END) AS guest_fee,

MAX(CASE WHEN f.fee_type='Host service fee' THEN f.amount END) AS host_fee,

ROUND(b.total_price + MAX(CASE WHEN f.fee_type='Guest service fee' THEN f.amount END),2) AS guest_pays,

ROUND(b.total_price - MAX(CASE WHEN f.fee_type='Host service fee' THEN f.amount END),2) AS host_gets

FROM booking b

JOIN fee f ON f.booking_id = b.booking_id

GROUP BY b.booking_id, b.total_price

ORDER BY b.booking_id

LIMIT 10;

-- -----

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	booking_id	total_price	guest_fee	host_fee	guest_pays	host_gets
▶	BKG001	1130.90	135.71	33.93	1266.61	1096.97
	BKG002	876.24	105.15	26.29	981.39	849.95
	BKG003	1227.20	147.26	36.82	1374.46	1190.38
	BKG004	427.12	51.25	12.81	478.37	414.31
	BKG005	403.97	48.48	12.12	452.45	391.85
	BKG006	1794.31	215.32	53.83	2009.63	1740.48
	BKG007	1112.79	133.53	33.38	1246.32	1079.41
	BKG008	1212.75	145.53	36.38	1358.28	1176.37
	BKG009	448.40	53.81	13.45	502.21	434.95
	BKG010	1161.92	139.43	34.86	1301.35	1127.06

Revenue Per Host

Total released payouts by host.

52

-- -----

53

-- TEST CASE 3: revenue per host (released payouts only).

54

-- -----

55

• SELECT hu.username AS host,

56

COUNT(DISTINCT b.booking_id) AS bookings,

57

ROUND(SUM(CASE WHEN po.status='RELEASED' THEN po.amount ELSE 0 END),2) AS released_payout

58

FROM host h

59

JOIN `user` hu ON h.user_id = hu.user_id

60

JOIN listing l ON l.host_id = h.host_id

61

JOIN booking b ON b.listing_id = l.listing_id

62

JOIN payout po ON po.booking_id= b.booking_id

63

GROUP BY h.host_id, hu.username

64

ORDER BY released_payout DESC

65

LIMIT 10;

66

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	host	bookings	released_payout
▶	liam.jansen1	3	2411.11
	harper.novak17	1	1971.20
	jonas.papadopoulos18	1	1328.53
	noah.dubois3	1	1190.38
	lucas.byrne8	1	1176.37
	mason.gruber10	1	1127.06
	amelia.hansen13	1	865.14
	emma.muller2	1	849.95
	charlotte.korhonen15	1	770.96
	nora.horvat19	1	726.07

Bookings & Revenue by Geography

Bookings and revenue by country and city.

67

-- -----

68

-- TEST CASE 4: bookings and revenue by country and city

69

-- (booking -> listing -> address -> city -> country).

70

-- -----

71

•

SELECT co.country_name, ci.city_name,

72

COUNT(b.booking_id) AS bookings,

73

ROUND(SUM(b.total_price),2) AS revenue

74

FROM booking b

75

JOIN listing l ON b.listing_id = l.listing_id

76

JOIN address a ON l.address_id = a.address_id

77

JOIN city ci ON a.city_id = ci.city_id

78

JOIN country co ON ci.country_id = co.country_id

79

GROUP BY co.country_id, ci.city_id

80

ORDER BY revenue DESC

81

LIMIT 10;

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	country_name	city_name	bookings	revenue
▶	United States	New York	2	2322.48
	Austria	Vienna	1	2032.17
	France	Lyon	1	1794.31
	Switzerland	Zurich	1	1369.62
	United Kingdom	Manchester	1	1326.95
	Germany	Berlin	1	1227.20
	Spain	Barcelona	1	1212.75
	Italy	Milan	1	1161.92
	Netherlands	Amsterdam	1	1130.90
	Spain	Madrid	1	1112.79

Top Guests by Amount Paid

Top guests by total amount paid.

83

-- -----

84

-- TEST CASE 5: top guests by total amount actually paid.

85

-- -----

86

• SELECT gu.username AS guest,

87

COUNT(p.payment_id) AS payments,

88

ROUND(SUM(p.amount),2) AS total_paid

89

FROM payment p

90

JOIN booking b ON p.booking_id = b.booking_id

91

JOIN guest g ON b.guest_id = g.guest_id

92

JOIN `user` gu ON g.user_id = gu.user_id

93

WHERE p.status = 'PAID'

94

GROUP BY g.guest_id, gu.username

95

ORDER BY total_paid DESC

96

LIMIT 10;

97

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	guest	payments	total_paid
▶	ines.bakker21	2	2783.96
	timo.nikolaou37	1	2276.03
	alma.maric38	1	1533.97
	pavel.bernard23	1	1374.46
	klara.kelly28	1	1358.28
	aoife.berger30	1	1301.35
	anton.brown27	1	1246.32
	tom.ricci43	1	1083.82
	milan.larsen33	1	998.93
	marta.weber22	1	981.39

Ternary Relationship - Reviews

A review linking a booking to a reviewer and a reviewee.

```
98  -- -----
99  -- TEST CASE 6: TERNARY relationship - Review links a Booking to a
100 -- reviewer (User) and a reviewee (User). Joins review + booking + user twice.
101  -- -----
102 • SELECT r.review_id, b.booking_id,
103        ru.username AS reviewer, re.username AS reviewee,
104        r.overall_rating, r.review_text
105 FROM review r
106 JOIN booking b ON r.booking_id = b.booking_id
107 JOIN `user` ru ON r.reviewer_id = ru.user_id
108 JOIN `user` re ON r.reviewee_id = re.user_id
109 ORDER BY r.review_id
110 LIMIT 10;
111
112  -- -----
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

	review_id	booking_id	reviewer	reviewee	overall_rating	review_text
▶	REV001	BKG001	ines.bakker21	liam.jansen1	4.5	Host was responsive and helpful.
	REV002	BKG002	marta.weber22	emma.muller2	4.5	Exactly as described, lovely place.
	REV003	BKG003	pavel.bernard23	noah.dubois3	3.5	Host was responsive and helpful.
	REV004	BKG005	hugo.bianchi25	ava.rossi5	5.0	Comfortable and quiet, would book again.
	REV005	BKG008	klara.kelly28	lucas.byrne8	4.5	Exactly as described, lovely place.
	REV006	BKG009	bjorn.wouters29	mia.peeters9	3.5	Exactly as described, lovely place.
	REV007	BKG010	aoife.berger30	mason.gruber10	4.5	Smooth check-in, great value.
	REV008	BKG011	sven.karlsson31	isabella.meier11	3.5	Host was responsive and helpful.
	REV009	BKG012	lea.olsen32	ethan.andersson12	5.0	Comfortable and quiet, would book again.
	REV010	BKG013	milan.larsen33	amelia.hansen13	3.5	Smooth check-in, great value.

Average Score per Review Category

Average score per review category.

112
113
114
115
116
117
118
119
120
121
122
123
124

```
-- -----  
-- TEST CASE 7: average score per review category (review -> reviewrating  
-- -> reviewcategory).  
-- -----  
SELECT rc.cat_name,  
       ROUND(AVG(rr.score),2) AS avg_score,  
       COUNT(*) AS num_ratings  
FROM reviewrating rr  
JOIN reviewcategory rc ON rr.cat_id = rc.cat_id  
GROUP BY rc.cat_id, rc.cat_name  
ORDER BY avg_score DESC  
LIMIT 10;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	cat_name	avg_score	num_ratings
▶	Check-in	4.67	3
	Accuracy	4.60	5
	Wi-Fi quality	4.60	5
	Cleanliness	4.50	4
	Overall experience	4.33	6
	Kitchen quality	4.25	4
	Hospitality	4.17	6
	Comfort	4.14	7
	Safety	4.13	8
	Ambience	4.00	2

Average Guest Rating per Listing

Average guest rating per listing.

125
126
127
128 •
129
130
131
132
133
134
135
136
137

```
-- -----  
-- TEST CASE 8: average guest rating per listing (from reviews).  
-- -----  
SELECT l.title,  
       ROUND(AVG(r.overall_rating),2) AS avg_rating,  
       COUNT(r.review_id) AS reviews  
FROM review r  
JOIN booking b ON r.booking_id = b.booking_id  
JOIN listing l ON b.listing_id = l.listing_id  
GROUP BY l.listing_id, l.title  
ORDER BY avg_rating DESC  
LIMIT 10;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	title	avg_rating	reviews
▶	Romantic attic nest	5.00	1
	Historic townhouse gem	5.00	2
	Cosy city-centre apartment	4.75	2
	Bright loft with skyline view	4.75	2
	Spacious family villa	4.75	2
	Rustic countryside cottage	4.75	2
	Green rooftop hideaway	4.75	2
	Designer duplex downtown	4.50	2
	Luxury chalet with hot tub	4.50	1
	Boutique guest suite	4.50	2

Most-Wishlisted Listings

Most-wishlisted listings (many-to-many).

138

-- -----

139

-- TEST CASE 9: most-wishlisted listings (M:N wishlist <-> listing).

140

-- -----

141

•

SELECT l.title, COUNT(wi.wishlist_id) AS times_wishlisted

142

FROM wishlistitem wi

143

JOIN listing l ON wi.listing_id = l.listing_id

144

GROUP BY l.listing_id, l.title

145

ORDER BY times_wishlisted DESC

146

LIMIT 10;

147

148

-- -----

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	title	times_wishlisted
▶	Cosy city-centre apartment	2
	Bright loft with skyline view	2
	Charming canal house	2
	Modern studio near park	2
	Spacious family villa	2
	Quiet garden retreat	2
	Stylish penthouse suite	2
	Rustic countryside cottage	2
	Sunny balcony flat	2
	Designer duplex downtown	2

Amenity Popularity

Amenity popularity (many-to-many).

148

-- -----

149

-- TEST CASE 10: amenity popularity (M:N listing <-> amenity).

150

-- -----

151

•

SELECT am.amenity_name, COUNT(la.listing_id) AS listings_offering

152

FROM listingamenity la

153

JOIN amenity am ON la.amenity_id = am.amenity_id

154

GROUP BY am.amenity_id, am.amenity_name

155

ORDER BY listings_offering DESC

156

LIMIT 10;

157

158

-- -----

Result Grid

Filter Rows:

Exports

Wrap Cell Content:

Fetch rows:

	amenity_name	listings_offering
▶	Air conditioning	3
	Balcony	3
	Hair dryer	3
	Coffee maker	3
	Wi-Fi	3
	Dishwasher	3
	Gym	3
	Elevator	3
	Fire extinguisher	3
	First aid kit	3

Recursive Relationship - Connections

A user's social connections.

158

-- -----

159

-- TEST CASE 11: RECURSIVE relationship - a user's social connections

160

-- (userconnection joins user to user).

161

-- -----

162

•

SELECT u1.username AS user, u2.username AS connected_to, uc.connected_since

163

FROM userconnection uc

164

JOIN `user` u1 ON uc.user_id = u1.user_id

165

JOIN `user` u2 ON uc.connected_user_id = u2.user_id

166

ORDER BY u1.username

167

LIMIT 10;

168

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

Fetch rows:

	user	connected_to	connected_since
▶	alma.maric38	lea.olsen32	2025-09-22
	anton.brown27	nora.horvat19	2025-02-13
	ava.rossi5	mason.gruber10	2025-12-21
	ava.rossi5	ines.bakker21	2025-10-14
	charlotte.korhonen15	milan.larsen33	2025-09-27
	elijah.silva6	elena.costa26	2025-05-24
	elijah.silva6	klara.kelly28	2025-02-08
	emma.muller2	elijah.silva6	2025-12-17
	emma.muller2	ines.bakker21	2025-03-26
	ethan.andersson12	mia.peeters9	2025-07-18

Users as Both Host & Guest

Users who act as both host and guest.

169

-- -----

170

-- TEST CASE 12: users who act as BOTH host and guest

171

-- (a user having a row in host AND in guest).

172

-- -----

173

• SELECT u.user_id, u.username, u.role

174

FROM `user` u

175

JOIN host h ON h.user_id = u.user_id

176

JOIN guest g ON g.user_id = u.user_id

177

ORDER BY u.user_id;

178

179

-- -----

Result Grid

 Filter Rows:

Export: 

Wrap Cell Content: 

	user_id	username	role
▶	USR041	nikola.fischer41	BOTH
	USR042	sara.moreau42	BOTH
	USR043	tom.ricci43	BOTH

Payment Status Summary

Payment status summary (data-quality overview).

179

-- -----

180

-- TEST CASE 13: payment status summary (data-quality overview).

181

-- -----

182

•

SELECT status, COUNT(*) AS num_payments, ROUND(SUM(amount),2) AS total_amount

183

FROM payment

184

GROUP BY status

185

ORDER BY num_payments DESC;

186

187

-- End of queries.sql

188

189

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	status	num_payments	total_amount
▶	PAID	21	20319.03
	REFUNDED	2	2332.03
	FAILED	1	2009.63

Summary & Conclusion

- 29-table normalised database fully implemented in MySQL.
- All relationships and business rules enforced with constraints.
- 20+ rows per table forming complete, consistent scenarios.
- 13 test cases confirm the design works end to end.
- Ready for the finalisation phase.

SQL-Based Database Management System : airbnb_datamart

Phase 3 Finalisation Abstract | DLBDSPBDM01 | Kashish Paresh Gada | Matric. No. 102202657

Project Overview

This document summarises the finalised airbnb_datamart project, a relational database management system built in MySQL to simulate an Airbnb-style accommodation booking platform. The system covers the full booking lifecycle: user registration, property listing, reservation, payment, host payout, fee calculation, reviews and social messaging. Three user roles are modelled under the User entity: Hosts (list properties, manage availability and house rules), Guests (browse, book, pay and review) and users holding both roles simultaneously. The platform enforces a real-world fee structure: a 12% guest service fee and a 3% host service fee per booking.

Development Process Summary

Development followed a structured three-phase approach. The Conception phase produced a requirements specification and a relational Entity Relationship Model in crow's-foot notation covering 25 entities and 4 junction tables. The Development phase produced three validated SQL files: schema.sql (29 tables with all constraints), inserts.sql (minimum 20 rows per table, end-to-end business scenarios), and queries.sql (13 test-case queries). A combined complete_scenario.sql was also produced. All files were executed and verified on MySQL 8.0.46 with exit code 0 and zero errors.

Data Population Strategy

All 29 tables contain a minimum of 20 rows, totalling approximately 944 rows. Data was designed as coherent end-to-end scenarios: each of the 24 bookings has a matching payment, payout and two fee rows. The 18 completed bookings each have at least one review with per-category ratings. The 43 user records are split into 20 host-only, 20 guest-only and 3 dual-role users.

Constraint Implementation and Data Integrity

Data integrity is enforced at database level through four constraint types. PRIMARY KEY: every table has a named 'pk_' constraint with readable VARCHAR(50) IDs. FOREIGN KEY: all 37 ERD relationships enforced with appropriate ON UPDATE CASCADE / ON DELETE CASCADE or RESTRICT actions. UNIQUE: 16 constraints on identity fields and 1:1 relationship. CHECK: 19 business rules including ratings 1-5, check_out after check_in, non-negative amounts, valid lat/long ranges, and no self-reviews, self-messages or self-connections. InnoDB engine with MySQL 8.0.16+ is required for CHECK enforcement.

System Metadata

Property	Value
Database Name	airbnb_datamart

Total Tables	29 (25 entities + 4 junction tables)
Engine / Charset	InnoDB / utf8mb4
PRIMARY KEY constraints	29
FOREIGN KEY constraints	37
UNIQUE constraints	16
CHECK constraints	19
Total Rows (all tables)	~944
Total Database Size	1.11 MB
DBMS / Tool	MySQL 8.0.46 / MySQL Workbench 8.0

Row count per table:

Table	Rows	Table	Rows
country	20	city	24
address	30	user	43
host	23	guest	23
socialaccount	26	paymentmethod	26
payoutmethod	26	propertytype	24
roomtype	20	cancellationpolicy	20
amenity	25	reviewcategory	20
listing	24	photo	48
houserule	30	availabilitycalendar	48
listingamenity	72	booking	24
payment	24	payout	24
fee	48	review	30
reviewrating	90	message	30
wishlist	24	wishlistitem	48
userconnection	30		

Final Deliverables

- 01-Concept: Phase 1 PDF - requirements specification, ERD (crow's-foot, 29 tables), data dictionary
- 02-Development: Phase 2 presentation PDF (22 slides), SQL files (schema, inserts, queries, complete_scenario)
- 03-Finalisation: This abstract (PDF), README installation manual (.md + PDF)
- Combined PDF: single document with all content from all three phases including screenshots
- ZIP archive: Gada-Kashish_102202657_DLBDSBDM01_Data_Mart_Submission_all.zip